Interrupciones externas en el ESP32

Un botón que **avisa solo** por interrupción. La ISR debe ser **corta**, estar en **IRAM**, usar variables `volatile` y el **patrón bandera**: así ningún evento se pierde.

Hardware · Software · código · Regla · decisión · Error · ojo · ◇ decisión

1 Del botón al loop()

mientras corre la ISR, loop() está congelado

1 — Se presiona el botón `BOTÓN`

Con `INPUT_PULLUP` el pin pasa de 1 a 0: un flanco **FALLING**.

```
pin 21: 1 → 0
```

2 — El periférico GPIO lo detecta `GPIO + CPU`

Lo vigila **en hardware**, aunque `loop()` esté en un `delay()`. La CPU salta a la ISR.

3 — Corre la ISR `isrBoton` `ISR`

Antirrebote ≥ 50 ms, sube la bandera y sale: **corta**.

```
void IRAM_ATTR isrBoton()
```

Si falta → **Sin `IRAM_ATTR`**: Con la caché apagada la CPU no encuentra la ISR: el chip se cae (de vez en cuando).

4 — Bandera `volatile` `VARIABLE`

`alerta = true`: arriba significa «pasó algo».

```
volatile bool alerta
```

Si falta → **Sin `volatile`**: `loop()` nunca ve el cambio: falla el paso bandera → loop().

5 — `loop()` hace el trabajo pesado `LOOP()`

Primero baja la bandera, después trabaja (Serial, sirena...). Tarde, **pero no se pierde**.

2 ¿FALLING, RISING o CHANGE?

nivel = la puerta está abierta · flanco = se acaba de abrir

? ¿Cuándo debe actuar la ISR?

Con `INPUT_PULLUP` la lógica es invertida: suelto = 1, presionado = 0.

Al presionar

Con **pull-up** presionar baja la señal; con pull-down, la sube.

```
pull-up → FALLING
pull-down → RISING
```

Al soltar

Con pull-up, soltar sube la señal de 0 a 1.

```
RISING
```

En ambos

La ISR corre **dos veces por toque**.

```
CHANGE
```

`PREGUNTA TÍPICA`

Botón con `INPUT_PULLUP`, alarma al presionar → **FALLING**: el pin se une a GND y la señal **cae** de 1 a 0.

3 ¿Lleva volatile?

no importa dónde se escribe, sino cuántas partes la tocan

1. **¿La usa la ISR?** — No → **No necesita `volatile`**
2. Sí → **¿La usa también `loop()`?** — No → **No necesita `volatile`**
3. Sí → `volatile`
   Solo hace eso: leer la RAM cada vez. **No** la hace más rápida, más segura ni atómica.

```
volatile bool alerta = false;
```

VARIABLE	ISR	LOOP	VOLATILE
`alerta`	escribe	lee	Sí
`pulsos`	escribe	lee	Sí
`tUltimo`	lee/escribe	—	No
`total`	—	lee/escribe	No

POR QUÉ

El compilador no ve la ISR: piensa «nadie cambia `alerta` aquí, la copio una vez y no vuelvo a la RAM». El LED nunca enciende y **no hay error de compilación**.

ERROR TRAICIONERO

Con un `Serial.println` en el loop, el compilador suele volver a leer la RAM y «parece» que `volatile` sobraba. Sin el println, o con otra optimización, deja de funcionar. Si cumple la regla: **va volatile**.

4 Caché apagada → caída

raro pero real: «a veces falla y no sé por qué»

1. **El ESP32 escribe en la flash**
2. **La caché se apaga un momento**
3. **Llega una interrupción** — y la ISR está en la flash
4. **La CPU no puede leerla** — el chip se cae

TODA ISR LLEVA IRAM_ATTR. SIEMPRE.

`void IRAM_ATTR isrBoton() { … }` copia la ISR a la **IRAM** (instrucciones). La DRAM (*Data RAM*) guarda los datos, como `alerta`.

✔ SÍ VA EN LA ISR

- Subir una bandera
- Guardar el tiempo: `millis()`
- Sumar 1 a un contador
- `digitalWrite` (es rápido)

✘ NO VA EN LA ISR

- `Serial.print` / `println`
- `delay()`
- Cálculos largos
- Esperar a que algo pase

PINES QUE ADMITEN INTERRUPCIÓN (ESP32; EN EL UNO SOLO D2 Y D3)

PINES	¿SIRVEN?	CUIDADO
16, 17, 18, 19, 21, 22, 23, 25, 26, 27, 32, 33	Sí	Los recomendados (AP4: botón en GPIO21).
34, 35, 36, 39 (solo entrada)	Con cuidado	Sin pull-up interna: usar **10 kΩ a 3,3 V**; si no, el pin flota y da alarmas falsas.
0, 2, 5, 12, 15 (strapping)	Evitar	Si están presionados al encender, arranca mal.
6 a 11 (flash)	No	Los usa la memoria del programa.

CÓDIGO AP4: ALERTA DOMÉSTICA

```
const uint8_t PIN_BOTON = 21;
const uint8_t PIN_LED   = 19;
const uint32_t T_REBOTE = 50;           // ms
volatile bool alerta = false;           // ISR y loop
uint32_t tUltimo = 0;                   // solo la ISR
void IRAM_ATTR isrBoton() {
  uint32_t ahora = millis();
  if (ahora - tUltimo >= T_REBOTE) {
    alerta = true;
    tUltimo = ahora;
  }
}
void setup() {
  Serial.begin(115200);
  pinMode(PIN_LED, OUTPUT);
  pinMode(PIN_BOTON, INPUT_PULLUP);
  attachInterrupt(digitalPinToInterrupt(PIN_BOTON), isrBoton, FALLING);
}
void loop() {
  if (alerta) {
    alerta = false;
    digitalWrite(PIN_LED, !digitalRead(PIN_LED));
    Serial.println("¡Alerta! Puerta abierta");
  }
  delay(2000);                          // loop ocupado: el aviso NO se pierde
}
```

attachInterrupt: ISR sin paréntesis · una sola vez, en setup() · después de pinMode().

RESPUESTAS · CONTROL DE APRENDIZAJE

01 *INPUT_PULLUP y alarma al presionar: ¿RISING, FALLING o CHANGE?*
FALLING. Con pull-up el pin está en 1; al presionar se une a GND y la señal **cae** de 1 a 0. RISING sería al soltar; CHANGE, dos veces por toque.

02 *¿Cuáles llevan volatile: `pulsos` (ISR suma, loop lee), `tUltimo` (solo ISR), `total` (solo loop)?*
Solo `pulsos`: la tocan la ISR **y** el loop.

03 *`Serial.println` dentro de la ISR y el ESP32 se reinicia con «Guru Meditation Error». ¿Qué pasa?*
La ISR es **demasiado larga** y el **watchdog** reinicia el chip. Corrección: **patrón bandera** (la ISR solo hace `alerta = true`; el `println` va en `loop()`).

04 *Sin antirrebote, 10 toques: ¿exactamente 10, menos o más?*
Más de 10. Cada toque rebota y produce varios flancos de bajada; cada uno dispara la ISR.

05 *Timer a 1 MHz, alarma 500 000, autorrecarga: ¿cada cuánto corre la ISR?*
500 000 / 1 000 000 = **0,5 s**: **2 veces por segundo**.